

Compendio Estratégico: Orquestación y Encadenamiento del Ecosistema Merci

Compendio | Vol. 1

mercedev.es — 2026-05-27 | Fase Epic 7 - Fase 1

El Desafío (Síntoma)

A medida que el ecosistema DevSecOps de Merci evolucionaba, la cantidad de scripts independientes en Python creció hasta alcanzar los 32 agentes especializados. Gestionar manualmente la ejecución de optimizadores de imágenes, compiladores CSS, validadores SEO, motores de generación estática (SSG) y asistentes de inteligencia artificial generaba una inmensa fricción operativa.

El reto arquitectónico consistía en encadenar estas herramientas para formar *Pipelines* automáticos (Cadenas de Suministro), garantizando que los agentes se pasaran el testigo (contexto) sin generar acoplamientos rígidos ni colisiones, manteniendo la regla innegociable de Cero Dependencias Externas.

La Maniobra (Lógica)

La solución adoptada fue agrupar los agentes mediante el patrón de **Agent Chaining** y la **Orquestación Dirigida por Estados** (Spec-Driven), donde el archivo `.md` (y su YAML Frontmatter) actúa como la única fuente de verdad (SSOT). El sistema se divide en cuatro grandes cadenas de montaje:

1. Cadena de Construcción y Auditoría (Pipeline Maestro)

Encabezada por `merci-total.py`, esta cadena se ejecuta de forma secuencial y aplica el patrón *Fail-Fast* (si un agente falla, el resto se aborta):

- Preparación Visual:** `merci-optimizer.py` convierte imágenes nuevas a WebP y `merci-styles.py` compila la arquitectura SASS.
- Construcción Estática:** `merci-publish.py` compila Markdown a HTML/PDF y delega en `merci-sync-pages.py` la propagación del diseño base a las páginas independientes.

3. **Auditoría (QA Estricto):** `merci-sitemap.py` genera el mapa XML. A continuación, el escudo activo `merci-audit.py` escanea secretos, SEO y sintaxis. Finalmente, `merci-linkcheck.py` actúa como DAST verificando que no existan enlaces rotos.

2. Cadena de Contenidos y DevRel (Agent Chaining IA)

Esta es la cadena cognitiva impulsada por Inteligencia Artificial (Ollama / Gemini), que automatiza la creación y difusión:

1. **Redacción Técnica:** `merci-librarian.py` transforma notas en crudo en cuadernillos técnicos con formato YAML estricto, ubicándolos en `incubacion/`.
2. **Re-empaquetado Social (COPE):** El Bibliotecario invoca directamente a `merci-blogger.py`, pasándole el testigo. Este segundo agente lee el cuadernillo y redacta un *post* comercial para LinkedIn y el Blog, inyectando automáticamente las URLs canónicas.
3. **Difusión Asíncrona:** El documento queda retenido en el Buffer Social. La autora puede monitorearlo con `merci-queue.py` y, tras validarlo, `merci-linkedin.py` dispara el contenido mediante la API OIDC a la red profesional.

3. Cadena de E-commerce y Publicación Headless

Gobernada por la sincronización bidireccional contra el servidor WordPress aislado:

1. **Enrutamiento Inteligente:** `merci-promote.py` saca el archivo de la incubadora, lee su campo `tema` y lo mueve a la raíz de la Tienda, el Blog o la Biblioteca.
2. **Sincronización:** `merci-wp.py` (para entradas) y `merci-shop.py` (para productos WooCommerce) envían los archivos físicos hacia las APIs REST del CMS, utilizando *slugs* para resolver los IDs dinámicamente y evitando duplicados.

4. Cadena de SRE, Hardening y Release (Distribución)

Cadena asíncrona dedicada a la monitorización y la infraestructura de nube:

1. **Observabilidad:** `merci-sre.py` extrae métricas del repositorio y del YAML para inyectarlas en Prometheus y Grafana de manera pasiva.
2. **Resiliencia:** `merci-chaos.py` ataca el código para poner a prueba las defensas de `merci-audit.py`.
3. **Distribución y Clon Efímero:** `merci-release.py` y `merci-showcase.py` clonan el proyecto en una carpeta temporal, ejecutan la guillotina destructiva de `merci-init.py` (purga de datos personales y telemetría) y despliegan el código inmaculado hacia repositorios públicos o subdominios de demostración.

Mapa de Decisiones y Flujo de Trabajo

El siguiente esquema ilustra la orquestación de los agentes y cómo se delegan el trabajo secuencialmente según la etapa del proyecto:

```
[ EVENTO O ACCIÓN EN EL ECOSISTEMA ]
|
|→ 1. FLUJO DE REDACCIÓN Y CONTENIDO (Agent Chaining)
|   |→ merci-librarian.py ---> Da formato a nota cruda
|   |   |→ merci-blogger.py ---> Extrae post promocional
|   |       |→ merci-promote.py ---> Enruta a Biblioteca/Blog
|   |           |→ merci-queue.py ---> Buffer Social
|   |               |→ [ Aprobación ] ---> merci-linkedin.py (Publica
OIDC)
|
|→ 2. FLUJO DE FONDO Y SRE (Asíncrono)
|   |→ Watchers ---> merci-assets-watcher.py / merci-watcher.py
|   |→ Telemetría -> merci-sre.py / merci-telemetry.py
|   |→ Seguridad ---> merci-hardening.py / merci-chaos.py
|   |→ IA Docs ---> merci-brain.py / merci-ssot.py / merci-
glosario.py
```

```

|      └─ Recovery    ---> merci-backup.py
|
|→ 3. PIPELINE MAESTRO (Build & QA)
|      └─ merci-total.py
|          └─ 1. Assets: merci-optimizer.py / merci-styles.py
|          └─ 2. Build:  merci-publish.py / merci-sync-pages.py
|          └─ 3. SEO:    merci-sitemap.py
|          └─ 4. QA:     merci-audit.py / merci-drift.py / merci-
linkcheck.py
|              └─ [ FALLO QA ] ---> merci-auto-fix.py (GitHub Actions)
|              └─ [ ÉXITO QA ] ---> merci-commit.py (Sello Atómico)
|
|→ 4. DESPLIEGUE Y DISTRIBUCIÓN
|    └─ A Producción: merci-completo.py
|        └─ merci-deploy.py ---> merci-wp.py / merci-shop.py --->
merci-extract-metrics.py
|
|    └─ Al Boilerplate: merci-release.py
|        └─ merci-init.py (Guillotina DLP)
|            └─ merci-showcase.py (Clon Efímero / Demo Showcase)

```

El Aprendizaje / Deuda Técnica

El éxito de la orquestación masiva de agentes recae en el principio de Separación de Responsabilidades (SRP). Ningún script conoce el funcionamiento interno del otro; solo conocen el formato de entrada (Markdown/YAML) y el estado del sistema de archivos. Si en el futuro es necesario reemplazar el optimizador de imágenes o cambiar el modelo de IA local, el resto de la cadena de montaje permanecerá intacto e inquebrantable.



En resumen (Merci Explica):

Imagina que nuestro proyecto es una inmensa fábrica de coches. En lugar de tener un solo robot gigantesco que intente ensamblar el coche entero (lo que sería lento y propenso a romperse), tenemos 32 pequeños robots especializados.

Uno pinta la carrocería (SASS), otro verifica que los frenos funcionen (Auditor), y otro redacta el manual de instrucciones del conductor (Inteligencia Artificial). La verdadera magia no son los robots por separado, sino la **cinta transportadora** (el YAML Frontmatter) que asegura que cada coche pase por las estaciones en el orden perfecto, entregando un producto seguro y ultrarrápido al final de la línea.